Sri Lanka Institute of Information Technology



Lab Submission
<Lab sheet No 5>

<IT26100802>

<Wickramasinghe V.I.>

Discrete Mathematics | IT1160

B.Sc. (Hons) in Information Technology

Part 01

01.

[1]:
```
import itertools

actors = ["A", "B", "C"]

schedules = list(itertools.permutations(actors))

for schedule in schedules:
    print(schedule)

print("Total schedules:", len(schedules))
```

```
('A', 'B', 'C')
('A', 'C', 'B')
('B', 'A', 'C')
('B', 'C', 'A')
('C', 'A', 'B')
('C', 'B', 'A')
Total schedules: 6
```

[2]:
```
print("5th schedule:", schedules[4])
```

```
5th schedule: ('C', 'A', 'B')
```

02.

[3]:
```
import itertools

letters = ["a", "b", "c", "d"]
digits = ["0", "1", "2", "3", "4"]

letter_pairs = list(itertools.permutations(letters, 2))
digit_triples = list(itertools.permutations(digits, 3))

passwords = []

for letter_pair in letter_pairs:
    for digit_triple in digit_triples:
        password = "".join(letter_pair) + "".join(digit_triple)
        passwords.append(password)

print("Total unique passwords:", len(passwords))
print("First 10 passwords:")

for password in passwords[:10]:
    print(password)
```

```
Total unique passwords: 720
First 10 passwords:
ab012
ab013
ab014
ab021
ab023
ab024
ab031
ab032
ab034
ab041
```

03.

[4]:
```
import math

right_moves = 3
up_moves = 3
total_moves = right_moves + up_moves

number_of_paths = math.factorial(total_moves) // (
    math.factorial(right_moves) * math.factorial(up_moves)
)

print("Number of unique paths:", number_of_paths)
```

```
Number of unique paths: 20
```

[5]:
```
path = ["R", "R", "R", "U", "U", "U"]

print("One valid path:", path)
```

```
One valid path: ['R', 'R', 'R', 'U', 'U', 'U']
```

04.

[6]:
```
import itertools

employees = ["A", "B", "C", "D"]

committees = list(itertools.combinations(employees, 2))

for committee in committees:
    print(committee)

print("Total committees:", len(committees))
```

```
('A', 'B')
('A', 'C')
('A', 'D')
('B', 'C')
('B', 'D')
('C', 'D')
Total committees: 6
```

05.

[7]:
```
import itertools

flowers = ["Rose", "Lily", "Tulip"]

bouquets = list(itertools.product(flowers, repeat=3))

for bouquet in bouquets:
    print(bouquet)

print("Total bouquets:", len(bouquets))
```

```
('Rose', 'Rose', 'Rose')
('Rose', 'Rose', 'Lily')
('Rose', 'Rose', 'Tulip')
('Rose', 'Lily', 'Rose')
('Rose', 'Lily', 'Lily')
('Rose', 'Lily', 'Tulip')
('Rose', 'Tulip', 'Rose')
('Rose', 'Tulip', 'Lily')
('Rose', 'Tulip', 'Tulip')
('Lily', 'Rose', 'Rose')
('Lily', 'Rose', 'Lily')
('Lily', 'Rose', 'Tulip')
('Lily', 'Lily', 'Rose')
('Lily', 'Lily', 'Lily')
('Lily', 'Lily', 'Tulip')
('Lily', 'Tulip', 'Rose')
('Lily', 'Tulip', 'Lily')
('Lily', 'Tulip', 'Tulip')
('Tulip', 'Rose', 'Rose')
('Tulip', 'Rose', 'Lily')
('Tulip', 'Rose', 'Tulip')
('Tulip', 'Lily', 'Rose')
('Tulip', 'Lily', 'Lily')
('Tulip', 'Lily', 'Tulip')
('Tulip', 'Tulip', 'Rose')
('Tulip', 'Tulip', 'Lily')
('Tulip', 'Tulip', 'Tulip')
Total bouquets: 27
```

[8]:
```
different_bouquets = []

for bouquet in bouquets:
    if len(set(bouquet)) == 3:
        different_bouquets.append(bouquet)

for bouquet in different_bouquets:
    print(bouquet)

print("Total bouquets with different flowers:", len(different_bouquets))
```

```
('Rose', 'Lily', 'Tulip')
```

Part-02

01.

[9]:
```python
import math

total_books = 7
red_books = 3
green_books = 2

unique_permutations = math.factorial(total_books) // (
    math.factorial(red_books) * math.factorial(green_books)
)

print("Unique permutations:", unique_permutations)
```

```
Unique permutations: 420
```

b)

[10]:

```
import math

red_block = "RRR"
green_block = "GG"
blue_book = "B"
orange_book = "O"

items = [red_block, green_block, blue_book, orange_book]

arrangements = list(itertools.permutations(items))

print("Number of arrangements:", len(arrangements))

for arrangement in arrangements:
    print(arrangement)
```

```
Number of arrangements: 24
('RRR', 'GG', 'B', 'O')
('RRR', 'GG', 'O', 'B')
('RRR', 'B', 'GG', 'O')
('RRR', 'B', 'O', 'GG')
('RRR', 'O', 'GG', 'B')
('RRR', 'O', 'B', 'GG')
('GG', 'RRR', 'B', 'O')
('GG', 'RRR', 'O', 'B')
('GG', 'B', 'RRR', 'O')
('GG', 'B', 'O', 'RRR')
('GG', 'O', 'RRR', 'B')
('GG', 'O', 'B', 'RRR')
('B', 'RRR', 'GG', 'O')
('B', 'RRR', 'O', 'GG')
('B', 'GG', 'RRR', 'O')
('B', 'GG', 'O', 'RRR')
('B', 'O', 'RRR', 'GG')
('B', 'O', 'GG', 'RRR')
('O', 'RRR', 'GG', 'B')
('O', 'RRR', 'B', 'GG')
('O', 'GG', 'RRR', 'B')
('O', 'GG', 'B', 'RRR')
('O', 'B', 'RRR', 'GG')
('O', 'B', 'GG', 'RRR')
```

02.

[11]:
```
import itertools

digits = [1, 2, 3, 4, 5]

numbers = list(itertools.permutations(digits, 3))

print("Total three-digit numbers:", len(numbers))
```

```
Total three-digit numbers: 60
```

b)

[12]:
```
greater_than_500 = []

for number in numbers:
    if number[0] == 5:
        greater_than_500.append(number)

print("Numbers greater than 500:", len(greater_than_500))

for number in greater_than_500:
    print("".join(map(str, number)))
```

```
Numbers greater than 500: 12
512
513
514
521
523
524
531
532
534
541
542
543
```

03.

[13]:
```
import math

paths_to_obstacle = math.comb(4, 2)
paths_from_obstacle = math.comb(2, 1)

paths_through_obstacle = paths_to_obstacle * paths_from_obstacle

print("Paths through (2,2):", paths_through_obstacle)
```

```
Paths through (2,2): 12
```

[14]:
```
total_paths = math.comb(6, 3)

paths_avoiding_obstacle = total_paths - paths_through_obstacle

print("Total paths:", total_paths)
print("Paths through (2,2):", paths_through_obstacle)
print("Paths avoiding (2,2):", paths_avoiding_obstacle)
```

```
Total paths: 20
Paths through (2,2): 12
Paths avoiding (2,2): 8
```

04.

[15]:
```python
import math

total_servers = 15
servers_to_activate = 11

number_of_selections = math.comb(
    total_servers, servers_to_activate
)

print("Number of possible selections:", number_of_selections)
```

```
Number of possible selections: 1365
```

[16]:
```python
import math

computer_ways = math.comb(8, 6)
storage_ways = math.comb(5, 4)
backup_ways = math.comb(2, 1)

total_selections = computer_ways * storage_ways * backup_ways

print("Ways to select computer servers:", computer_ways)
print("Ways to select storage servers:", storage_ways)
print("Ways to select backup servers:", backup_ways)
print("Total valid selections:", total_selections)
```

```
Ways to select computer servers: 28
Ways to select storage servers: 5
Ways to select backup servers: 2
Total valid selections: 280
```

05.

[16]:
```python
import math

computer_ways = math.comb(8, 6)
storage_ways = math.comb(5, 4)
backup_ways = math.comb(2, 1)

total_selections = computer_ways * storage_ways * backup_ways

print("Ways to select computer servers:", computer_ways)
print("Ways to select storage servers:", storage_ways)
print("Ways to select backup servers:", backup_ways)
print("Total valid selections:", total_selections)
```

```
Ways to select computer servers: 28
Ways to select storage servers: 5
Ways to select backup servers: 2
Total valid selections: 280
```

[17]:
```python
import math

total_vms = 10
number_of_types = 4

number_of_combinations = math.comb(
    total_vms + number_of_types - 1,
    number_of_types - 1
)

print("Total VM allocation combinations:", number_of_combinations)
```

```
Total VM allocation combinations: 286
```